

```
// ===== 文件: vue-cycling-app/src/main.js | 技术栈: JavaScript (ES6+) =====
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import '@fortawesome/fontawesome-free/css/all.min.css'
import './assets/styles/base.css'

// 在开发环境中导入数据库测试
if (import.meta.env.DEV) {
  import('./utils/dbTest.js');
}

const app = createApp(App)
app.use(router)
app.mount('#app')

// ===== 文件: vue-cycling-app/src/App.vue | 技术栈: Vue 3 + JavaScript =====
<script setup>
import { onMounted } from 'vue';
import Navbar from './components/Navbar.vue';
import Footer from './components/Footer.vue';
import '@fortawesome/fontawesome-free/css/all.min.css';
import './assets/styles/base.css';

onMounted(() => {
  // 在组件挂载后执行的逻辑
  document.title = '灵境骑行 - 探索城市周边最美骑行路线';
});
</script>

<template>
  <div class="app">
    <Navbar />
    <router-view />
    <Footer />
  </div>
</template>

<style>
/* 全局样式 */
* {
  box-sizing: border-box;
}

/* 确保页面占满整个视口高度 */
.app {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

/* 确保主要内容区域可以伸展填充空间 */
```

```

main {
  flex: 1;
}
</style>
// ===== 文件: vue-cycling-app/src/router/index.js | 技术栈: Vue 3 + Vue Router =====
import { createRouter, createWebHistory } from 'vue-router'
import { isAuthenticated } from '../utils/auth.js'

const router = createRouter({
  history: createWebHistory(),
  scrollBehavior(to, from, savedPosition) {
    // 如果有保存的位置（比如浏览器前进后退），则返回到保存的位置
    if (savedPosition) {
      return savedPosition
    }
    // 如果路由有hash，则滚动到对应的元素
    if (to.hash) {
      return {
        el: to.hash,
        behavior: 'smooth'
      }
    }
    // 默认滚动到页面顶部
    return { top: 0, behavior: 'smooth' }
  },
  routes: [
    {
      path: '/',
      name: 'Home',
      component: () => import('../views/Home.vue')
    },
    {
      path: '/login',
      name: 'Login',
      component: () => import('../views/Login.vue')
    },
    {
      path: '/register',
      name: 'Register',
      component: () => import('../views/Register.vue')
    },
    {
      path: '/feedback',
      name: 'Feedback',
      component: () => import('../views/Feedback.vue'),
      meta: { requiresAuth: true }
    },
    {
      path: '/help',
      name: 'Help',
      component: () => import('../views/Help.vue')
    }
  ]
})

```

```

    },
    {
      path: '/vr',
      name: 'VR',
      component: () => import('../views/VR.vue'),
      meta: { requiresAuth: true }
    },
    {
      path: '/route-planning',
      name: 'RoutePlanning',
      component: () => import('../views/RoutePlanning.vue'),
      meta: { requiresAuth: true }
    },
    {
      path: '/safety-guide',
      name: 'SafetyGuide',
      component: () => import('../views/SafetyGuide.vue')
    },
    {
      path: '/maintenance-guide',
      name: 'MaintenanceGuide',
      component: () => import('../views/MaintenanceGuide.vue')
    },
    {
      path: '/event-calendar',
      name: 'EventCalendar',
      component: () => import('../views/EventCalendar.vue')
    },
    {
      path: '/equipment',
      name: 'Equipment',
      component: () => import('../views/Equipment.vue')
    },
    {
      path: '/intangible-heritage-map',
      name: 'IntangibleHeritageMap',
      component: () => import('../views/IntangibleHeritageMap.vue')
    }
  ]
})

// 全局前置守卫 - 检查登录状态
router.beforeEach((to, from, next) => {
  // 检查目标路由是否需要认证
  if (to.matched.some(record => record.meta.requiresAuth)) {
    // 检查用户是否已登录
    if (!isAuthenticated()) {
      // 未登录, 跳转到登录页面
      next({
        path: '/login',
        query: { redirect: to.fullPath } // 保存原始目标路径, 登录后可以跳转回去
      })
    }
  }
})

```

```

    })
  } else {
    // 已登录，继续访问
    next()
  }
} else {
  // 不需要认证的路由，直接访问
  next()
}
})

export default router

// ===== 文件: vue-cycling-app/src/utils/auth.js | 技术栈: JavaScript (ES6+) =====
// 认证工具函数

/**
 * 检查用户是否已登录
 * @returns {boolean} 是否已登录
 */
export function isAuthenticated() {
  const token = localStorage.getItem('accessToken');
  const userInfo = localStorage.getItem('userInfo');
  return !! (token && userInfo);
}

/**
 * 获取当前用户信息
 * @returns {object|null} 用户信息对象或null
 */
export function getCurrentUser() {
  try {
    const userInfo = localStorage.getItem('userInfo');
    return userInfo ? JSON.parse(userInfo) : null;
  } catch (error) {
    console.error('解析用户信息失败:', error);
    return null;
  }
}

/**
 * 获取访问令牌
 * @returns {string|null} 访问令牌或null
 */
export function getAccessToken() {
  return localStorage.getItem('accessToken');
}

/**
 * 获取刷新令牌
 * @returns {string|null} 刷新令牌或null
 */

```

```
export function getRefreshToken() {
  return localStorage.getItem('refreshToken');
}

/**
 * 清除认证信息（登出）
 */
export function clearAuth() {
  localStorage.removeItem('accessToken');
  localStorage.removeItem('refreshToken');
  localStorage.removeItem('userInfo');
}

/**
 * 设置认证信息
 * @param {string} accessToken 访问令牌
 * @param {string} refreshToken 刷新令牌
 * @param {object} userInfo 用户信息
 */
export function setAuth(accessToken, refreshToken, userInfo) {
  localStorage.setItem('accessToken', accessToken);
  localStorage.setItem('refreshToken', refreshToken);
  localStorage.setItem('userInfo', JSON.stringify(userInfo));
}

// ===== 文件: vue-cycling-app/src/config/api.js | 技术栈: JavaScript (ES6+) =====
// API配置文件
// 统一管理所有后端服务的URL配置

// 环境配置
const isDev = import.meta.env.DEV;
const isProd = import.meta.env.PROD;

// 后端服务配置
// 开发模式: Java 用 /api (由 Vite proxy 转发)
// 生产模式: Java 用绝对 URL (Flask serve 静态文件时相对路径会进 Python 后端)
export const API_CONFIG = {
  // Java后端服务 (主要API服务)
  JAVA_BACKEND: {
    baseUrl: isDev ? '/api' : 'http://localhost:8080/api',
    timeout: 10000,
    description: 'Java后端 - 主要API服务、用户认证、路线规划等'
  },

  // Python后端服务 (爬虫服务)
  PYTHON_BACKEND: {
    baseUrl: 'http://localhost:5000/api',
    timeout: 15000,
    description: 'Python后端 - 爬虫服务、设备管理等'
  },

  // 路线规划服务 (Java后端的特定配置)
```

```
ROUTE_PLANNING: {
  baseUrl: isDev ? '/api' : 'http://localhost:8080/api',
  timeout: 30000, // 路线规划可能需要更长时间
  description: 'Java后端 - 路线规划专用配置'
}
};
```

// 默认请求头配置

```
export const DEFAULT_HEADERS = {
  'Content-Type': 'application/json',
  'Accept': 'application/json'
};
```

// API端点配置

```
export const API_ENDPOINTS = {
  // 认证相关
  AUTH: {
    LOGIN: '/auth/login',
    REGISTER: '/auth/register',
    LOGOUT: '/auth/logout',
    REFRESH: '/auth/refresh'
  },
```

// 用户相关

```
USER: {
  PROFILE: '/user/profile',
  UPDATE_PROFILE: '/user/profile'
},
```

// 路线相关

```
ROUTES: {
  LIST: '/routes',
  CREATE: '/routes',
  DETAIL: (id) => `/routes/${id}`,
  UPDATE: (id) => `/routes/${id}`,
  DELETE: (id) => `/routes/${id}`,
  SEARCH: '/routes/search'
},
```

// 路线规划相关 (Java后端)

```
ROUTE_PLANNING: {
  PLAN: '/route/plan',
  QUICK_PLAN: '/route/quick-plan',
  GEOCODE: '/route/geocode',
  REVERSE_GEOCODE: '/route/reverse-geocode',
  TRANSPORT_MODES: '/route/transport-modes',
  STRATEGIES: '/route/strategies',
  HEALTH: '/route/health'
},
```

// 骑行记录相关

```

RECORDS: {
    LIST: '/records',
    CREATE: '/records',
    STATS: '/records/stats'
},

// 反馈相关
FEEDBACK: {
    SUBMIT: '/feedback',
    LIST: '/feedback'
},

// 数据库测试相关
DATABASE: {
    HEALTH: '/health',
    TEST: '/test',
    TABLES: '/tables',
    CYCLING_ROUTES: '/cycling-routes'
},

// Python后端 - 爬虫服务
CRAWLER: {
    STATUS: '/crawler/status',
    START: '/crawler/start',
    STOP: '/crawler/stop'
},

// Python后端 - 设备管理
EQUIPMENT: {
    LIST: '/equipment',
    SEARCH: '/equipment/search',
    DETAIL: (id) => `/equipment/${id}`,
    CATEGORIES: '/equipment/categories',
    CREATE: '/equipment',
    UPDATE: (id) => `/equipment/${id}`,
    DELETE: (id) => `/equipment/${id}`
},

// Python后端 - 数据分析（专为图表组件提供）
ANALYSIS: {
    TRENDS: '/analysis/trends',
    COMPETITION: '/analysis/competition',
    PRICE_ALERTS: '/analysis/price-alerts',
    RECOMMENDATIONS: '/analysis/recommendations',
    PRICE_HISTORY: (equipmentId) => `/analysis/price-history/${equipmentId}`,
    BRAND_MARKET_SHARE: '/analysis/brand-market-share',
    PRICE_DISTRIBUTION: '/analysis/price-distribution',
    CLEAR_CACHE: '/analysis/clear-cache'
}
};

```

```
// 错误码配置
export const ERROR_CODES = {
  NETWORK_ERROR: 'ERR_NETWORK',
  TIMEOUT: 'ECONNABORTED',
  UNAUTHORIZED: 401,
  FORBIDDEN: 403,
  NOT_FOUND: 404,
  INTERNAL_SERVER_ERROR: 500
};

// 错误消息配置
export const ERROR_MESSAGES = {
  [ERROR_CODES.NETWORK_ERROR]: '网络连接失败，请检查网络设置',
  [ERROR_CODES.TIMEOUT]: '请求超时，请稍后重试',
  [ERROR_CODES.UNAUTHORIZED]: '登录已过期，请重新登录',
  [ERROR_CODES.FORBIDDEN]: '权限不足，无法访问该资源',
  [ERROR_CODES.NOT_FOUND]: '请求的资源不存在',
  [ERROR_CODES.INTERNAL_SERVER_ERROR]: '服务器内部错误，请稍后重试'
};

export default API_CONFIG;
// ===== 文件: vue-cycling-app/src/services/api.js | 技术栈: JavaScript (ES6+) =====
import axios from 'axios';
import { API_CONFIG, DEFAULT_HEADERS, API_ENDPOINTS, ERROR_CODES, ERROR_MESSAGES } from '@config/api.js';

// 创建axios实例 - Java后端
const api = axios.create({
  baseURL: API_CONFIG.JAVA_BACKEND.baseURL,
  timeout: API_CONFIG.JAVA_BACKEND.timeout,
  headers: DEFAULT_HEADERS
});

// 创建Python后端axios实例
const pythonApi = axios.create({
  baseURL: API_CONFIG.PYTHON_BACKEND.baseURL,
  timeout: API_CONFIG.PYTHON_BACKEND.timeout,
  headers: DEFAULT_HEADERS
});

// Python后端响应拦截器
pythonApi.interceptors.response.use(
  response => {
    return response.data;
  },
  error => {
    console.error('Python API请求错误:', error);
    return Promise.reject(error);
  }
);

// 请求拦截器
```

```

api.interceptors.request.use(
  config => {
    // 可以在这里添加token等认证信息
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  error => {
    return Promise.reject(error);
  }
);

// 响应拦截器
api.interceptors.response.use(
  response => {
    return response.data;
  },
  error => {
    console.error('API请求错误:', error);

    // 处理不同的错误状态码
    if (error.response) {
      const status = error.response.status;
      const message = ERROR_MESSAGES[status] || error.response.data?.message || '未知错误';

      switch (status) {
        case ERROR_CODES.UNAUTHORIZED:
          // 未授权，清除token并跳转到登录页
          localStorage.removeItem('token');
          localStorage.removeItem('accessToken');
          localStorage.removeItem('refreshToken');
          localStorage.removeItem('userInfo');
          window.location.href = '/login';
          break;
        case ERROR_CODES.FORBIDDEN:
          console.error('权限不足:', message);
          break;
        case ERROR_CODES.NOT_FOUND:
          console.error('资源不存在:', message);
          break;
        case ERROR_CODES.INTERNAL_SERVER_ERROR:
          console.error('服务器错误:', message);
          break;
        default:
          console.error('请求失败:', message);
      }
    } else if (error.request) {
      const errorCode = error.code;
      const message = ERROR_MESSAGES[errorCode] || '网络错误，请检查网络连接';
    }
  }
);

```

```

        console.error(message);
    } else {
        console.error('请求配置错误:', error.message);
    }

    return Promise.reject(error);
}
);

// API服务类
class ApiService {
    // 用户相关API
    static user = {
        // 用户登录
        login: (credentials) => api.post(API_ENDPOINTS.AUTH.LOGIN, credentials),

        // 用户注册
        register: (userData) => api.post(API_ENDPOINTS.AUTH.REGISTER, userData),

        // 获取用户信息
        getProfile: () => api.get(API_ENDPOINTS.USER.PROFILE),

        // 更新用户信息
        updateProfile: (userData) => api.put(API_ENDPOINTS.USER.UPDATE_PROFILE, userData),

        // 用户登出
        logout: () => api.post(API_ENDPOINTS.AUTH.LOGOUT),

        // 刷新token
        refreshToken: () => api.post(API_ENDPOINTS.AUTH.REFRESH)
    };

    // 骑行路线相关API
    static routes = {
        // 获取所有路线
        getAll: () => api.get('/routes'),

        // 根据ID获取路线
        getById: (id) => api.get(`/routes/${id}`),

        // 创建新路线
        create: (routeData) => api.post('/routes', routeData),

        // 更新路线
        update: (id, routeData) => api.put(`/routes/${id}`, routeData),

        // 删除路线
        delete: (id) => api.delete(`/routes/${id}`),

        // 搜索路线
        search: (query) => api.get(`/routes/search?q=${encodeURIComponent(query)}`)
    }
}

```

```
};

// 路线规划相关API (Java后端)
static routePlanning = {
  // 创建路线规划专用axios实例 (需要更长超时时间)
  _api: axios.create({
    baseURL: API_CONFIG.ROUTE_PLANNING.baseURL,
    timeout: API_CONFIG.ROUTE_PLANNING.timeout,
    headers: DEFAULT_HEADERS
  }),

  // 完整路线规划
  planRoute: function(routeData) {
    return ApiService.routePlanning._api.post(API_ENDPOINTS.ROUTE_PLANNING.PLAN, routeData);
  },

  // 快速路线规划
  quickPlan: function(origin, destination, transportMode, city = '石家庄') {
    const params = new URLSearchParams({
      origin,
      destination,
      transportMode,
      city
    });
    return ApiService.routePlanning._api.get(`${API_ENDPOINTS.ROUTE_PLANNING.QUICK_PLAN}?${params}`);
  },

  // 地理编码 (地址转坐标)
  geocode: function(address, city = '石家庄') {
    const params = new URLSearchParams({ address, city });
    return api.get(`${API_ENDPOINTS.ROUTE_PLANNING.GEOCODE}?${params}`);
  },

  // 逆地理编码 (坐标转地址)
  reverseGeocode: function(longitude, latitude) {
    const params = new URLSearchParams({ longitude, latitude });
    return api.get(`${API_ENDPOINTS.ROUTE_PLANNING.REVERSE_GEOCODE}?${params}`);
  },

  // 获取支持的交通方式
  getTransportModes: function() {
    return api.get(API_ENDPOINTS.ROUTE_PLANNING.TRANSPORT_MODES);
  },

  // 获取路线策略
  getStrategies: function() {
    return api.get(API_ENDPOINTS.ROUTE_PLANNING.STRATEGIES);
  },

  // 健康检查
  healthCheck: function() {
```

```
    return api.get(API_ENDPOINTS.ROUTE_PLANNING.HEALTH);
  }
};

// 骑行记录相关API
static records = {
  // 获取用户的骑行记录
  getUserRecords: () => api.get('/records'),

  // 创建骑行记录
  create: (recordData) => api.post('/records', recordData),

  // 获取骑行统计
  getStats: () => api.get('/records/stats')
};

// 反馈相关API
static feedback = {
  // 提交反馈
  submit: (feedbackData) => api.post(API_ENDPOINTS.FEEDBACK.SUBMIT, feedbackData),

  // 获取反馈列表（管理员）
  getAll: () => api.get(API_ENDPOINTS.FEEDBACK.LIST)
};

// Python后端 - 装备相关API（已合并到下方的equipment定义中）

// Python后端 - 爬虫服务API
static crawler = {
  // 获取爬虫状态
  getStatus: () => pythonApi.get(API_ENDPOINTS.CRAWLER.STATUS),

  // 启动爬虫
  start: (config = {}) => pythonApi.post(API_ENDPOINTS.CRAWLER.START, config),

  // 停止爬虫
  stop: () => pythonApi.post(API_ENDPOINTS.CRAWLER.STOP)
};

// Python后端 - 设备管理API（合并所有equipment相关方法）
static equipment = {
  // 搜索装备
  search: (params) => pythonApi.get(API_ENDPOINTS.EQUIPMENT.SEARCH, { params }),

  // 获取装备分类
  getCategories: () => pythonApi.get(API_ENDPOINTS.EQUIPMENT.CATEGORIES),

  // 获取装备详情
  getDetail: (id) => pythonApi.get(API_ENDPOINTS.EQUIPMENT.DETAIL(id)),

  // 获取装备列表
```

```

getList: (params) => pythonApi.get(API_ENDPOINTS.EQUIPMENT.LIST, { params }),

// 获取设备列表（别名）
getAll: (params = {}) => pythonApi.get(API_ENDPOINTS.EQUIPMENT.LIST, { params }),

// 创建新设备
create: (equipmentData) => pythonApi.post(API_ENDPOINTS.EQUIPMENT.CREATE, equipmentData),

// 更新设备信息
update: (id, equipmentData) => pythonApi.put(API_ENDPOINTS.EQUIPMENT.UPDATE(id), equipmentData),

// 删除设备
delete: (id) => pythonApi.delete(API_ENDPOINTS.EQUIPMENT.DELETE(id))
};

// 数据库测试相关API
static database = {
  healthCheck: () => api.get(API_ENDPOINTS.DATABASE.HEALTH),
  test: () => api.get(API_ENDPOINTS.DATABASE.TEST),
  getTables: () => api.get(API_ENDPOINTS.DATABASE.TABLES),
  getCyclingRoutes: (params = {}) => api.get(API_ENDPOINTS.DATABASE.CYCLING_ROUTES, { params })
};

// Python后端 - 数据分析API（供图表组件调用）
static analysis = {
  // 获取装备价格趋势分析
  getEquipmentTrends: (days = 30) =>
    pythonApi.get(API_ENDPOINTS.ANALYSIS.TRENDS, { params: { days } }),

  // 获取单个商品多平台价格历史（图表专用）
  getPriceHistory: (equipmentId, days = 30) =>
    pythonApi.get(API_ENDPOINTS.ANALYSIS.PRICE_HISTORY(equipmentId), { params: { days } }),

  // 获取品牌市场份额（图表专用）
  getBrandMarketShare: (category = null) =>
    pythonApi.get(API_ENDPOINTS.ANALYSIS.BRAND_MARKET_SHARE, { params: category ? { category } : {} }),

  // 获取价格分布统计（图表专用）
  getPriceDistribution: (category = null) =>
    pythonApi.get(API_ENDPOINTS.ANALYSIS.PRICE_DISTRIBUTION, { params: category ? { category } : {} }),

  // 获取市场竞争分析
  getMarketCompetition: (category = null) =>
    pythonApi.get(API_ENDPOINTS.ANALYSIS.COMPETITION, { params: category ? { category } : {} }),

  // 获取价格预警
  getPriceAlerts: (threshold = 10.0) =>
    pythonApi.get(API_ENDPOINTS.ANALYSIS.PRICE_ALERTS, { params: { threshold } }),

  // 获取装备推荐
  getRecommendations: (params = {}) =>

```

[illegible]

```
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.web.cors.CorsConfigurationSource;
```

```
import java.util.List;
import java.util.Arrays;
```

```
/**
 * Spring Security配置类
 *
 * @author LJXZ Team
 * @version 1.0.0
 */
@Configuration
@EnableWebSecurity
@EnableConfigurationProperties(SecurityConfig.SecurityProperties.class)
@RequiredArgsConstructor
public class SecurityConfig {
```

```
    private final UserDetailsServiceImpl userDetailsService;
    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    private final SecurityProperties securityProperties;
    private final CorsConfigurationSource corsConfigurationSource;
```

```
    @ConfigurationProperties(prefix = "security")
    public static class SecurityProperties {
        private List<String> permitAll = Arrays.asList(
            "/auth/**",
            "/api/auth/**",
            "/public/**",
            "/health",
            "/route/health",
            "/api/route/health",
            "/test",
            "/api/test",
            "/tables",
            "/api/tables",
            "/cycling-routes/**",
            "/api/cycling-routes/**",
            "/feedback/**",
            "/api/feedback/**",
            "/actuator/**",
            "/swagger-ui/**",
```

```

        "/v3/api-docs/**"
    );

    public List<String> getPermitAll() {
        return permitAll;
    }

    public void setPermitAll(List<String> permitAll) {
        this.permitAll = permitAll;
    }
}

/**
 * 密码编码器
 */
@Bean
public BCryptPasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

/**
 * 认证提供者
 */
@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    authProvider.setUserDetailsService(userDetailsService);
    authProvider.setPasswordEncoder(passwordEncoder());
    return authProvider;
}

/**
 * 认证管理器
 */
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}

/**
 * 安全过滤器链
 */
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        // 禁用CSRF
        .csrf(AbstractHttpConfigurer::disable)

        // 配置CORS - 使用WebConfig中定义的corsConfigurationSource
        .cors(cors -> cors.configurationSource(corsConfigurationSource));
}

```

```
// 配置授权规则
.authorizeHttpRequests(authz -> {
    // 允许访问的路径
    String[] permitAllArray = securityProperties.getPermitAll().toArray(new String[0]);
    authz.requestMatchers(permitAllArray).permitAll();

    // 其他请求需要认证
    authz.anyRequest().authenticated();
})

// 配置会话管理
.sessionManagement(session ->
    session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
)

// 配置认证提供者
.authenticationProvider(authenticationProvider())

// 添加JWT过滤器
.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

return http.build();
}
}

// ===== 文件: java-backend/src/main/java/com/ljxz/cycling/config/JwtAuthenticationFilter.java | 技术栈: Java + Spri...
package com.ljxz.cycling.config;

import com.ljxz.cycling.service.UserDetailsServiceImpl;
import com.ljxz.cycling.util.JwtUtil;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

/**
 * JWT认证过滤器
 *
 * @author LJXZ Team
 * @version 1.0.0
 */
```

```

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;
    private final UserDetailsServiceImpl userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        final String authorizationHeader = request.getHeader("Authorization");

        String username = null;
        String jwt = null;

        // 检查Authorization头是否存在且以"Bearer "开头
        if (authorizationHeader != null && authorizationHeader.startsWith("Bearer ")) {
            jwt = authorizationHeader.substring(7);
            try {
                username = jwtUtil.getUsernameFromToken(jwt);
            } catch (Exception e) {
                logger.warn("JWT token解析失败: " + e.getMessage());
            }
        }

        // 如果用户名不为空且当前没有认证信息
        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            try {
                UserDetails userDetails = userDetailsService.loadUserByUsername(username);

                // 验证token
                if (jwtUtil.validateToken(jwt, userDetails)) {
                    UsernamePasswordAuthenticationToken authToken =
                        new UsernamePasswordAuthenticationToken(
                            userDetails, null, userDetails.getAuthorities());
                    authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            } catch (Exception e) {
                logger.warn("用户认证失败: " + e.getMessage());
            }
        }

        filterChain.doFilter(request, response);
    }
}

// ===== 文件: java-backend/src/main/java/com/ljxz/cycling/controller/AuthController.java | 技术栈: Java + Spring Bo...
package com.ljxz.cycling.controller;

```

```
import com.ljxz.cycling.dto.LoginRequest;
import com.ljxz.cycling.dto.RefreshTokenRequest;
import com.ljxz.cycling.dto.RegisterRequest;
import com.ljxz.cycling.service.AuthService;
import com.ljxz.cycling.service.UserService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

/**
 * 认证控制器
 *
 * @author LJXZ Team
 * @version 1.0.0
 */
@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor

public class AuthController {

    private final AuthService authService;
    private final UserService userService;

    /**
     * 用户登录
     */
    @PostMapping("/login")
    public ResponseEntity<Map<String, Object>> login(@Valid @RequestBody LoginRequest request) {
        Map<String, Object> result = authService.login(request.getUsernameOrEmail(), request.getPassword());

        if ((Boolean) result.get("success")) {
            return ResponseEntity.ok(result);
        } else {
            return ResponseEntity.badRequest().body(result);
        }
    }

    /**
     * 用户注册
     */
    @PostMapping("/register")
    public ResponseEntity<Map<String, Object>> register(@Valid @RequestBody RegisterRequest request) {
        // 验证密码确认
        if (!request.getPassword().equals(request.getConfirmPassword())) {

```

```

        Map<String, Object> result = new HashMap<>();
        result.put("success", false);
        result.put("message", "密码和确认密码不匹配");
        return ResponseEntity.badRequest().body(result);
    }

    // 构建全名
    String fullName = "";
    if (request.getFirstName() != null && request.getLastName() != null) {
        fullName = request.getFirstName() + " " + request.getLastName();
    } else if (request.getFirstName() != null) {
        fullName = request.getFirstName();
    } else if (request.getLastName() != null) {
        fullName = request.getLastName();
    }

    Map<String, Object> result = authService.register(
        request.getUsername(),
        request.getEmail(),
        request.getPassword(),
        fullName
    );

    if ((Boolean) result.get("success")) {
        return ResponseEntity.ok(result);
    } else {
        return ResponseEntity.badRequest().body(result);
    }
}

/**
 * 刷新Token
 */
@PostMapping("/refresh")
public ResponseEntity<Map<String, Object>> refreshToken(@Valid @RequestBody RefreshTokenRequest request) {
    Map<String, Object> result = authService.refreshToken(request.getRefreshToken());

    if ((Boolean) result.get("success")) {
        return ResponseEntity.ok(result);
    } else {
        return ResponseEntity.badRequest().body(result);
    }
}

/**
 * 用户登出
 */
@PostMapping("/logout")
public ResponseEntity<Map<String, Object>> logout() {
    // 由于使用JWT，登出主要在前端处理（删除token）

```

```
// 这里可以添加token黑名单逻辑
Map<String, Object> result = new HashMap<>();
result.put("success", true);
result.put("message", "登出成功");
return ResponseEntity.ok(result);
}

/**
 * 检查用户名是否可用
 */
@GetMapping("/check-username")
public ResponseEntity<Map<String, Object>> checkUsername(@RequestParam String username) {
    boolean exists = userService.existsByUsername(username);
    Map<String, Object> result = new HashMap<>();
    result.put("available", !exists);
    result.put("message", exists ? "用户名已存在" : "用户名可用");
    return ResponseEntity.ok(result);
}

/**
 * 检查邮箱是否可用
 */
@GetMapping("/check-email")
public ResponseEntity<Map<String, Object>> checkEmail(@RequestParam String email) {
    boolean exists = userService.existsByEmail(email);
    Map<String, Object> result = new HashMap<>();
    result.put("available", !exists);
    result.put("message", exists ? "邮箱已存在" : "邮箱可用");
    return ResponseEntity.ok(result);
}
}

// ===== 文件: java-backend/src/main/java/com/ljxz/cycling/service/AuthService.java | 技术栈: Java + Spring Boot (Se...
package com.ljxz.cycling.service;

import com.ljxz.cycling.entity.User;
import com.ljxz.cycling.util.JwtUtil;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import java.util.HashMap;
import java.util.Map;

/**
 * 认证服务类
 */
```

```

* @author LJXZ Team
* @version 1.0.0
*/
@Service
@RequiredArgsConstructor
public class AuthService {

    private final AuthenticationManager authenticationManager;
    private final UserService userService;
    private final JwtUtil jwtUtil;

    /**
     * 用户登录
     * @param usernameOrEmail 用户名或邮箱
     * @param password 密码
     * @return 登录结果
     */
    public Map<String, Object> login(String usernameOrEmail, String password) {
        try {
            // 认证用户
            Authentication authentication = authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(usernameOrEmail, password)
            );

            UserDetails userDetails = (UserDetails) authentication.getPrincipal();
            User user = (User) userDetails;

            // 生成token
            String accessToken = jwtUtil.generateToken(userDetails);
            String refreshToken = jwtUtil.generateRefreshToken(userDetails);

            Map<String, Object> result = new HashMap<>();
            result.put("success", true);
            result.put("message", "登录成功");
            result.put("accessToken", accessToken);
            result.put("refreshToken", refreshToken);
            result.put("user", getUserInfo(user));

            return result;
        } catch (BadCredentialsException e) {
            Map<String, Object> result = new HashMap<>();
            result.put("success", false);
            result.put("message", "用户名或密码错误");
            return result;
        } catch (Exception e) {
            Map<String, Object> result = new HashMap<>();
            result.put("success", false);
            result.put("message", "登录失败: " + e.getMessage());
            return result;
        }
    }
}

```

```

    }
}

/**
 * 用户注册
 * @param username 用户名
 * @param email 邮箱
 * @param password 密码
 * @param fullName 全名
 * @return 注册结果
 */
public Map<String, Object> register(String username, String email, String password, String fullName) {
    try {
        // 创建用户对象
        User user = new User();
        user.setUsername(username);
        user.setEmail(email);
        user.setPasswordHash(password); // 在UserService中会进行加密
        user.setFullName(fullName);

        // 注册用户
        User savedUser = userService.registerUser(user);

        Map<String, Object> result = new HashMap<>();
        result.put("success", true);
        result.put("message", "注册成功");
        result.put("user", getUserInfo(savedUser));

        return result;
    } catch (RuntimeException e) {
        Map<String, Object> result = new HashMap<>();
        result.put("success", false);
        result.put("message", e.getMessage());
        return result;
    } catch (Exception e) {
        Map<String, Object> result = new HashMap<>();
        result.put("success", false);
        result.put("message", "注册失败: " + e.getMessage());
        return result;
    }
}

/**
 * 刷新token
 * @param refreshToken 刷新token
 * @return 新的token
 */
public Map<String, Object> refreshToken(String refreshToken) {
    try {

```

```

        if (!jwtUtil.validateTokenFormat(refreshToken)) {
            Map<String, Object> result = new HashMap<>();
            result.put("success", false);
            result.put("message", "无效的刷新token");
            return result;
        }

        String username = jwtUtil.getUsernameFromToken(refreshToken);
        User user = userService.findByUsernameOrEmail(username)
            .orElseThrow(() -> new RuntimeException("用户不存在"));

        String newAccessToken = jwtUtil.generateToken(user);
        String newRefreshToken = jwtUtil.generateRefreshToken(user);

        Map<String, Object> result = new HashMap<>();
        result.put("success", true);
        result.put("message", "Token刷新成功");
        result.put("accessToken", newAccessToken);
        result.put("refreshToken", newRefreshToken);

        return result;
    } catch (Exception e) {
        Map<String, Object> result = new HashMap<>();
        result.put("success", false);
        result.put("message", "Token刷新失败: " + e.getMessage());
        return result;
    }
}

/**
 * 获取用户信息（不包含敏感信息）
 */
private Map<String, Object> getUserInfo(User user) {
    Map<String, Object> userInfo = new HashMap<>();
    userInfo.put("id", user.getId());
    userInfo.put("username", user.getUsername());
    userInfo.put("email", user.getEmail());
    userInfo.put("fullName", user.getFullName());
    userInfo.put("phone", user.getPhone());
    userInfo.put("avatarUrl", user.getAvatarUrl());
    userInfo.put("createdAt", user.getCreatedAt());
    return userInfo;
}
}

// ===== 文件: java-backend/src/main/java/com/ljxz/cycling/util/JwtUtil.java | 技术栈: Java + Spring Boot (JWT Utili...
package com.ljxz.cycling.util;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;

```

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

/**
 * JWT工具类
 *
 * @author LJXZ Team
 * @version 1.0.0
 */
@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private Long expiration;

    @Value("${jwt.refresh-expiration}")
    private Long refreshExpiration;

    /**
     * 获取密钥
     */
    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secret.getBytes());
    }

    /**
     * 从token中获取用户名
     */
    public String getUsernameFromToken(String token) {
        return getClaimFromToken(token, Claims::getSubject);
    }

    /**
     * 从token中获取过期时间
     */
    public Date getExpirationDateFromToken(String token) {
        return getClaimFromToken(token, Claims::getExpiration);
    }

    /**
```

```

    * 从token中获取指定声明
    */
    public <T> T getClaimFromToken(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = getAllClaimsFromToken(token);
        return claimsResolver.apply(claims);
    }

    /**
    * 从token中获取所有声明
    */
    private Claims getAllClaimsFromToken(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
    }

    /**
    * 检查token是否过期
    */
    private Boolean isTokenExpired(String token) {
        final Date expiration = getExpirationDateFromToken(token);
        return expiration.before(new Date());
    }

    /**
    * 生成token
    */
    public String generateToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        return createToken(claims, userDetails.getUsername(), expiration);
    }

    /**
    * 生成刷新token
    */
    public String generateRefreshToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("type", "refresh");
        return createToken(claims, userDetails.getUsername(), refreshExpiration);
    }

    /**
    * 创建token
    */
    private String createToken(Map<String, Object> claims, String subject, Long expiration) {
        return Jwts.builder()
            .setClaims(claims)
            .setSubject(subject)

```

```

        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() + expiration))
        .signWith(getSigningKey(), SignatureAlgorithm.HS256)
        .compact();
    }

/**
 * 验证token
 */
public Boolean validateToken(String token, UserDetails userDetails) {
    try {
        final String username = getUsernameFromToken(token);
        return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));
    } catch (JwtException | IllegalArgumentException e) {
        return false;
    }
}

/**
 * 验证token格式
 */
public Boolean validateTokenFormat(String token) {
    try {
        getAllClaimsFromToken(token);
        return true;
    } catch (JwtException | IllegalArgumentException e) {
        return false;
    }
}
}

# ===== 文件: python-backend/models/equipment.py | 技术栈: Python + Flask-SQLAlchemy =====
from datetime import datetime
from sqlalchemy import Index, DECIMAL
from sqlalchemy.dialects.mysql import BIGINT
from decimal import Decimal

def create_models(db):
    """创建所有模型类"""

    class EquipmentCategory(db.Model):
        """装备分类表"""
        __tablename__ = 'equipment_categories'

        id = db.Column(db.Integer, primary_key=True)
        name = db.Column(db.String(50), nullable=False, unique=True, comment='分类名称')
        name_en = db.Column(db.String(50), nullable=False, comment='英文名称')
        description = db.Column(db.Text, comment='分类描述')
        parent_id = db.Column(db.Integer, db.ForeignKey('equipment_categories.id'), comment='父分类ID')
        sort_order = db.Column(db.Integer, default=0, comment='排序')
        is_active = db.Column(db.Boolean, default=True, comment='是否启用')

```

```

created_at = db.Column(db.DateTime, default=datetime.utcnow)
updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

# 关系
children = db.relationship('EquipmentCategory', backref=db.backref('parent', remote_side=[id]))

def to_dict(self):
    return {
        'id': self.id,
        'name': self.name,
        'name_en': self.name_en,
        'description': self.description,
        'parent_id': self.parent_id,
        'sort_order': self.sort_order,
        'is_active': self.is_active,
        'children': [child.to_dict() for child in self.children if child.is_active]
    }

class Equipment(db.Model):
    """装备表 - 对齐真实数据库Schema"""
    __tablename__ = 'equipment'

    id = db.Column(BIGINT(unsigned=True), primary_key=True, autoincrement=True)
    name = db.Column(db.String(255), nullable=False, comment='装备名称')
    category_id = db.Column(db.Integer, db.ForeignKey('equipment_categories.id'), nullable=False, comment='分...
    subcategory = db.Column(db.String(50), comment='子分类')
    brand = db.Column(db.String(100), nullable=False, comment='品牌')
    model = db.Column(db.String(100), comment='型号')
    price = db.Column(DECIMAL(10, 2), comment='价格')
    original_price = db.Column(DECIMAL(10, 2), comment='原价')
    discount = db.Column(DECIMAL(3, 2), comment='折扣')
    description = db.Column(db.Text, comment='装备描述')
    detailed_description = db.Column(db.Text, comment='详细描述')
    specifications = db.Column(db.JSON, comment='规格参数')
    images = db.Column(db.JSON, comment='装备图片')
    tags = db.Column(db.JSON, comment='标签')
    weight = db.Column(db.Float, comment='重量')
    material = db.Column(db.String(200), comment='材质')
    color_options = db.Column(db.JSON, comment='颜色选项')
    size_options = db.Column(db.JSON, comment='尺寸选项')
    availability = db.Column(db.Boolean, default=True, nullable=False, comment='是否有货')
    stock = db.Column(db.Integer, comment='库存')
    rating = db.Column(DECIMAL(3, 2), comment='评分')
    review_count = db.Column(db.Integer, default=0, nullable=False, comment='评价数量')
    sales_count = db.Column(db.Integer, default=0, nullable=False, comment='销量')
    source = db.Column(db.String(50), nullable=False, comment='来源平台')
    source_url = db.Column(db.String(500), comment='来源链接')
    source_id = db.Column(db.String(100), comment='来源ID')
    last_updated = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

```

```

updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

# 关系
category_obj = db.relationship('EquipmentCategory', backref='equipment_items', lazy='select')

# 索引
__table_args__ = (
    Index('idx_equipment_category', 'category_id'),
    Index('idx_equipment_brand', 'brand'),
    Index('idx_equipment_rating', 'rating'),
    Index('idx_equipment_source', 'source'),
    Index('idx_equipment_availability', 'availability'),
    Index('idx_equipment_created', 'created_at'),
    Index('idx_equipment_name', 'name'),
)

@property
def is_active(self):
    """兼容属性：映射到 availability"""
    return self.availability

@property
def category(self):
    """兼容属性：返回分类名称字符串"""
    if self.category_obj:
        return self.category_obj.name
    return None

@property
def category_name(self):
    """返回分类名称"""
    if self.category_obj:
        return self.category_obj.name
    return None

@property
def rating_avg(self):
    """兼容属性：映射到 rating"""
    return float(self.rating) if self.rating else 0.0

@property
def rating_count(self):
    """兼容属性：映射到 review_count"""
    return self.review_count or 0

@property
def image_urls(self):
    """兼容属性：映射到 images"""
    return self.images

```

```

@property
def view_count(self):
    """兼容属性: 返回 sales_count 作为替代"""
    return self.sales_count or 0

@property
def favorite_count(self):
    """兼容属性"""
    return 0

@property
def status(self):
    """兼容属性"""
    return 'active' if self.availability else 'inactive'

def to_dict(self, include_prices=False, include_reviews=False):
    return {
        'id': self.id,
        'name': self.name,
        'brand': self.brand,
        'model': self.model,
        'category_id': self.category_id,
        'category': self.category,
        'subcategory': self.subcategory,
        'price': float(self.price) if self.price else None,
        'original_price': float(self.original_price) if self.original_price else None,
        'discount': float(self.discount) if self.discount else None,
        'description': self.description,
        'specifications': self.specifications,
        'images': self.images,
        'image_urls': self.images,
        'tags': self.tags,
        'weight': self.weight,
        'material': self.material,
        'color_options': self.color_options,
        'size_options': self.size_options,
        'availability': self.availability,
        'is_active': self.availability,
        'stock': self.stock,
        'avg_rating': float(self.rating) if self.rating else 0.0,
        'rating': float(self.rating) if self.rating else 0.0,
        'review_count': self.review_count,
        'rating_count': self.review_count,
        'sales_count': self.sales_count,
        'source': self.source,
        'source_url': self.source_url,
        'created_at': self.created_at.isoformat() if self.created_at else None,
        'updated_at': self.updated_at.isoformat() if self.updated_at else None
    }

```

```
# ===== 文件: python-backend/models/equipment.py | 技术栈: Python + Flask-SQLAlchemy =====

class EquipmentPrice(db.Model):
    """装备价格表"""
    __tablename__ = 'equipment_prices'

    id = db.Column(BIGINT(unsigned=True), primary_key=True)
    equipment_id = db.Column(BIGINT(unsigned=True), db.ForeignKey('equipment.id'), nullable=False)
    platform = db.Column(db.String(50), nullable=False, comment='平台名称')
    platform_url = db.Column(db.String(500), comment='商品链接')
    platform_id = db.Column(db.String(100), comment='平台商品ID')
    price = db.Column(DECIMAL(10, 2), nullable=False, comment='价格')
    original_price = db.Column(DECIMAL(10, 2), comment='原价')
    discount = db.Column(db.Float, comment='折扣')
    currency = db.Column(db.String(10), default='CNY', comment='货币')
    stock_status = db.Column(db.String(20), comment='库存状态')
    seller_name = db.Column(db.String(200), comment='卖家名称')
    seller_rating = db.Column(db.Float, comment='卖家评分')
    shipping_info = db.Column(db.JSON, comment='配送信息')
    promotion_info = db.Column(db.JSON, comment='促销信息')
    is_available = db.Column(db.Boolean, default=True, comment='是否有货')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    # 索引
    __table_args__ = (
        Index('idx_price_equipment', 'equipment_id'),
        Index('idx_price_platform', 'platform'),
        Index('idx_price_created', 'created_at'),
        Index('idx_price_equipment_platform', 'equipment_id', 'platform'),
    )

    def to_dict(self):
        return {
            'id': self.id,
            'equipment_id': self.equipment_id,
            'platform': self.platform,
            'platform_url': self.platform_url,
            'platform_id': self.platform_id,
            'price': float(self.price) if self.price else None,
            'original_price': float(self.original_price) if self.original_price else None,
            'discount': self.discount,
            'currency': self.currency,
            'stock_status': self.stock_status,
            'seller_name': self.seller_name,
            'seller_rating': self.seller_rating,
            'shipping_info': self.shipping_info,
            'promotion_info': self.promotion_info,
            'is_available': self.is_available,
            'created_at': self.created_at.isoformat() if self.created_at else None
        }

class EquipmentReview(db.Model):
```

```

"""装备评价表"""
__tablename__ = 'equipment_reviews'

id = db.Column(BIGINT(unsigned=True), primary_key=True)
equipment_id = db.Column(BIGINT(unsigned=True), db.ForeignKey('equipment.id'), nullable=False)
platform = db.Column(db.String(50), nullable=False, comment='平台名称')
platform_review_id = db.Column(db.String(100), comment='平台评价ID')
user_name = db.Column(db.String(100), comment='用户名')
user_avatar = db.Column(db.String(500), comment='用户头像')
rating = db.Column(db.Float, nullable=False, comment='评分')
title = db.Column(db.String(200), comment='评价标题')
content = db.Column(db.Text, comment='评价内容')
images = db.Column(db.JSON, comment='评价图片')
purchase_info = db.Column(db.JSON, comment='购买信息')
helpful_count = db.Column(db.Integer, default=0, comment='有用数')
reply_count = db.Column(db.Integer, default=0, comment='回复数')
is_verified = db.Column(db.Boolean, default=False, comment='是否认证购买')
review_date = db.Column(db.DateTime, comment='评价时间')
created_at = db.Column(db.DateTime, default=datetime.utcnow)

# 索引
__table_args__ = (
    Index('idx_review_equipment', 'equipment_id'),
    Index('idx_review_platform', 'platform'),
    Index('idx_review_rating', 'rating'),
    Index('idx_review_date', 'review_date'),
)

def to_dict(self):
    return {
        'id': self.id,
        'equipment_id': self.equipment_id,
        'platform': self.platform,
        'platform_review_id': self.platform_review_id,
        'user_name': self.user_name,
        'user_avatar': self.user_avatar,
        'rating': self.rating,
        'title': self.title,
        'content': self.content,
        'images': self.images,
        'purchase_info': self.purchase_info,
        'helpful_count': self.helpful_count,
        'reply_count': self.reply_count,
        'is_verified': self.is_verified,
        'review_date': self.review_date.isoformat() if self.review_date else None,
        'created_at': self.created_at.isoformat() if self.created_at else None
    }

return EquipmentCategory, Equipment, EquipmentPrice, EquipmentReview

```

模块级别的变量，将在app.py中设置

```

EquipmentCategory = None
Equipment = None
EquipmentPrice = None
EquipmentReview = None
// ===== 文件: java-backend/src/main/java/com/ljxz/cycling/controller/RouteController.java | 技术栈: Java + Spring B...
package com.ljxz.cycling.controller;

import com.ljxz.cycling.entity.RouteRequest;
import com.ljxz.cycling.entity.RouteResponse;
import com.ljxz.cycling.service.RouteService;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

import jakarta.validation.Valid;
import jakarta.validation.constraints.DecimalMax;
import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

/**
 * 路径规划控制器
 *
 * @author LJXZ Team
 * @version 1.0.0
 */
@Slf4j
@RestController
@RequestMapping("/")
@Validated
public class RouteController {

    @Autowired
    private RouteService routeService;

    /**
     * 路径规划接口
     *
     * @param request 路径规划请求
     * @return 路径规划响应
     */
    @PostMapping("/route/plan")
    public ResponseEntity<RouteResponse> planRoute(@Valid @RequestBody RouteRequest request) {
        log.info("收到路径规划请求: origin={}, destination={}, transportMode={}",
            request.getOrigin(), request.getDestination(), request.getTransportMode());

        RouteResponse response = routeService.planRoute(request);

        if (response.getStatus() == 1) {

```

```

        return ResponseEntity.ok(response);
    } else {
        return ResponseEntity.badRequest().body(response);
    }
}

/**
 * 快速路径规划接口（简化参数）
 *
 * @param origin 起点
 * @param destination 终点
 * @param transportMode 交通方式
 * @param city 城市（可选）
 * @return 路径规划响应
 */
@GetMapping("/route/quick-plan")
public ResponseEntity<RouteResponse> quickPlanRoute(
    @RequestParam @NotBlank(message = "起点不能为空") @Size(max = 200) String origin,
    @RequestParam @NotBlank(message = "终点不能为空") @Size(max = 200) String destination,
    @RequestParam @NotBlank(message = "交通方式不能为空") String transportMode,
    @RequestParam(required = false) String city) {

    log.info("收到快速路径规划请求: origin={}, destination={}, transportMode={}, city=",
        origin, destination, transportMode, city);

    RouteRequest request = RouteRequest.builder()
        .origin(origin)
        .destination(destination)
        .transportMode(transportMode)
        .city(city)
        .strategy(0)
        .showSteps(true)
        .build();

    RouteResponse response = routeService.planRoute(request);

    if (response.getStatus() == 1) {
        return ResponseEntity.ok(response);
    } else {
        return ResponseEntity.badRequest().body(response);
    }
}

/**
 * 地理编码接口 - 地址转坐标
 *
 * @param address 地址
 * @param city 城市（可选）
 * @return 坐标响应
 */
@GetMapping("/route/geocode")

```

```

public ResponseEntity<RouteResponse> geocode(
    @RequestParam @NotBlank(message = "地址不能为空") @Size(max = 200) String address,
    @RequestParam(required = false) String city) {

    log.info("收到地理编码请求: address={}, city={}", address, city);

    RouteResponse response = routeService.geocode(address, city);

    if (response.getStatus() == 1) {
        return ResponseEntity.ok(response);
    } else {
        return ResponseEntity.badRequest().body(response);
    }
}

/**
 * 逆地理编码接口 - 坐标转地址
 *
 * @param longitude 经度
 * @param latitude 纬度
 * @return 地址响应
 */
@GetMapping("/route/reverse-geocode")
public ResponseEntity<RouteResponse> reverseGeocode(
    @RequestParam @DecimalMin(value = "-180.0", message = "经度必须在-180到180之间")
        @DecimalMax(value = "180.0", message = "经度必须在-180到180之间") Double longitude,
    @RequestParam @DecimalMin(value = "-90.0", message = "纬度必须在-90到90之间")
        @DecimalMax(value = "90.0", message = "纬度必须在-90到90之间") Double latitude) {

    log.info("收到逆地理编码请求: longitude={}, latitude={}", longitude, latitude);

    RouteResponse response = routeService.reverseGeocode(longitude, latitude);

    if (response.getStatus() == 1) {
        return ResponseEntity.ok(response);
    } else {
        return ResponseEntity.badRequest().body(response);
    }
}

/**
 * 获取支持的交通方式
 *
 * @return 交通方式列表
 */
@GetMapping("/route/transport-modes")
public ResponseEntity<?> getTransportModes() {
    return ResponseEntity.ok(new Object() {
        public final String[] transportModes = {"driving", "walking", "transit", "riding"};
        public final Object[] descriptions = {
            new Object() {

```

```

        public final String mode = "driving";
        public final String name = "驾车";
        public final String description = "适用于汽车出行, 提供最优驾车路线";

        // 重写toString方法以使用字段
        @Override
        public String toString() {
            return mode + ": " + name + " - " + description;
        }
    },
    new Object() {
        public final String mode = "walking";
        public final String name = "步行";
        public final String description = "适用于步行出行, 提供步行路线";

        @Override
        public String toString() {
            return mode + ": " + name + " - " + description;
        }
    },
    new Object() {
        public final String mode = "transit";
        public final String name = "公交";
        public final String description = "适用于公共交通出行, 包括公交、地铁等";

        @Override
        public String toString() {
            return mode + ": " + name + " - " + description;
        }
    },
    new Object() {
        public final String mode = "riding";
        public final String name = "骑行";
        public final String description = "适用于自行车出行, 提供骑行路线";

        @Override
        public String toString() {
            return mode + ": " + name + " - " + description;
        }
    }
};

// 使用字段的方法
@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    for (Object desc : descriptions) {
        sb.append(desc.toString()).append("\n");
    }
    return sb.toString();
}

```

```

    }
    });
}

/**
 * 获取路径规划策略说明
 *
 * @return 策略说明
 */
@GetMapping("/route/strategies")
public ResponseEntity<?> getStrategies() {
    return ResponseEntity.ok(new Object() {
        public final Object[] strategies = {
            new Object() {
                public final Integer code = 0;
                public final String name = "速度优先";
                public final String description = "时间最短路线";

                @Override
                public String toString() {
                    return code + ": " + name + " - " + description;
                }
            },
            new Object() {
                public final Integer code = 1;
                public final String name = "费用优先";
                public final String description = "费用最少路线";

                @Override
                public String toString() {
                    return code + ": " + name + " - " + description;
                }
            },
            new Object() {
                public final Integer code = 2;
                public final String name = "距离优先";
                public final String description = "距离最短路线";

                @Override
                public String toString() {
                    return code + ": " + name + " - " + description;
                }
            },
            new Object() {
                public final Integer code = 3;
                public final String name = "不走快速路";
                public final String description = "避开快速路的路线";

                @Override
                public String toString() {

```

```

        return code + ": " + name + " - " + description;
    }
},
new Object() {
    public final Integer code = 4;
    public final String name = "躲避拥堵";
    public final String description = "避开拥堵路段";

    @Override
    public String toString() {
        return code + ": " + name + " - " + description;
    }
},
new Object() {
    public final Integer code = 5;
    public final String name = "多策略";
    public final String description = "综合速度、费用、距离的路线";

    @Override
    public String toString() {
        return code + ": " + name + " - " + description;
    }
}
};

// 使用字段的方法
@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    for (Object strategy : strategies) {
        sb.append(strategy.toString()).append("\n");
    }
    return sb.toString();
}
});
}

/**
 * 健康检查接口
 *
 * @return 服务状态
 */
@GetMapping({"/health", "/route/health"})
public ResponseEntity<?> health() {
    return ResponseEntity.ok(new Object() {
        public final String status = "UP";
        public final String message = "服务运行正常";
        public final String service = "灵境行者路径规划服务";
        public final String version = "1.0.0";
        public final long timestamp = System.currentTimeMillis();
    });
}

```

```

    });
}

/**
 * 测试接口
 *
 * @return 测试响应
 */
@GetMapping("/test")
public ResponseEntity<?> test() {
    return ResponseEntity.ok(new Object() {
        public final String status = "success";
        public final String message = "测试接口正常";
        public final String service = "灵境行者路径规划服务";
        public final long timestamp = System.currentTimeMillis();
    });
}

/**
 * 获取数据库表列表
 *
 * @return 数据库表列表
 */
@GetMapping("/tables")
public ResponseEntity<?> getTables() {
    log.info("获取数据库表列表");

    return ResponseEntity.ok(new Object() {
        public final boolean success = true;
        public final String[] tables = {
            "cycling_routes", "users", "route_history", "equipment",
            "weather_data", "poi_data", "route_points"
        };
        public final String message = "获取数据库表列表成功";
        public final long timestamp = System.currentTimeMillis();
    });
}

/**
 * 提交用户反馈
 *
 * @param feedbackData 反馈数据
 * @return 反馈提交结果
 */
@PostMapping("/feedback")
public ResponseEntity<?> submitFeedback(@RequestBody java.util.Map<String, Object> feedbackData) {
    log.info("收到用户反馈: {}", feedbackData);

    try {
        // 这里可以添加反馈数据的处理逻辑, 比如保存到数据库
    }
}

```

```

        java.util.Map<String, Object> response = new java.util.HashMap<>();
        response.put("success", true);
        response.put("message", "反馈提交成功");
        response.put("timestamp", System.currentTimeMillis());
        response.put("feedbackId", java.util.UUID.randomUUID().toString());

        return ResponseEntity.ok(response);
    } catch (Exception e) {
        log.error("处理反馈时发生错误", e);
        java.util.Map<String, Object> errorResponse = new java.util.HashMap<>();
        errorResponse.put("success", false);
        errorResponse.put("message", "反馈提交失败: " + e.getMessage());
        errorResponse.put("timestamp", System.currentTimeMillis());

        return ResponseEntity.status(500).body(errorResponse);
    }
}

/**
 * 获取骑行路线列表
 *
 * @param limit 限制返回数量
 * @return 骑行路线列表
 */
@GetMapping("/cycling-routes")
public ResponseEntity<?> getCyclingRoutes(
    @RequestParam(name = "limit", defaultValue = "10") @jakarta.validation.constraints.Min(value = 1, me...
    @jakarta.validation.constraints.Max(value = 100, message = "限制数量不能超过100") int limit) {

    log.info("获取骑行路线列表, 限制数量: {}", limit);

    // 模拟骑行路线数据
    java.util.List<Object> routes = new java.util.ArrayList<>();

    for (int i = 1; i <= Math.min(limit, 10); i++) {
        final int routeId = i; // 声明为final以在匿名内部类中使用
        routes.add(new Object() {
            public final int id = routeId;
            public final String name = "骑行路线 " + routeId;
            public final String distance = (routeId * 5.2) + " km";
            public final String duration = (routeId * 25) + " 分钟";
            public final String difficulty = routeId % 3 == 0 ? "困难" : routeId % 2 == 0 ? "中等" : "简单";
            public final String startPoint = "起点 " + routeId;
            public final String endPoint = "终点 " + routeId;
            public final String description = "这是一条优美的骑行路线, 适合" + (routeId % 3 == 0 ? "经验丰富" : "初学者") + "的骑行爱";
            public final long createdAt = System.currentTimeMillis();
        });
    }

    return ResponseEntity.ok(new Object() {

```

```

        public final boolean success = true;
        public final java.util.List<Object> data = routes;
        public final int total = routes.size();
        public final String message = "获取骑行路线列表成功";
    });
}

/**
 * 获取API使用说明
 *
 * @return API文档
 */
@GetMapping("/docs")
public ResponseEntity<?> getDocs() {
    return ResponseEntity.ok(new Object() {
        public final String title = "灵境行者路径规划API";
        public final String version = "1.0.0";
        public final String description = "提供路径规划、地理编码等服务";
        public final Object[] endpoints = {
            new Object() {
                public final String method = "POST";
                public final String path = "/api/route/plan";
                public final String description = "路径规划（完整参数）";
            },
            new Object() {
                public final String method = "GET";
                public final String path = "/api/route/quick-plan";
                public final String description = "快速路径规划（简化参数）";
            },
            new Object() {
                public final String method = "GET";
                public final String path = "/api/route/geocode";
                public final String description = "地理编码（地址转坐标）";
            },
            new Object() {
                public final String method = "GET";
                public final String path = "/api/route/reverse-geocode";
                public final String description = "逆地理编码（坐标转地址）";
            },
            new Object() {
                public final String method = "GET";
                public final String path = "/api/route/transport-modes";
                public final String description = "获取支持的交通方式";
            },
            new Object() {
                public final String method = "GET";
                public final String path = "/api/route/strategies";
                public final String description = "获取路径规划策略";
            }
        };
    });
}

```

```

    });
}
}

// ===== 文件: java-backend/src/main/java/com/ljxz/cycling/service/RouteService.java | 技术栈: Java + Spring Boot (S...
package com.ljxz.cycling.service;

import com.ljxz.cycling.entity.RouteRequest;
import com.ljxz.cycling.entity.RouteResponse;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.util.StringUtils;

import java.time.LocalDateTime;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.TimeUnit;

/**
 * 路径规划服务
 *
 * @author LJXZ Team
 * @version 1.0.0
 */
@Slf4j
@Service
public class RouteService {

    @Autowired
    private AmapService amapService;

    @Value("${route.default-city:石家庄}")
    private String defaultCity;

    @Value("${route.cache-expire-minutes:30}")
    private int cacheExpireMinutes;

    // 简单的内存缓存, 生产环境建议使用Redis
    private final ConcurrentHashMap<String, CacheEntry> routeCache = new ConcurrentHashMap<>();

    /**
     * 路径规划主方法
     *
     * @param request 路径规划请求
     * @return 路径规划响应
     */
    public RouteResponse planRoute(RouteRequest request) {
        long startTime = System.currentTimeMillis();

        try {

```

```
// 参数验证和预处理
validateAndPreprocessRequest(request);

// 检查缓存
String cacheKey = generateCacheKey(request);
CacheEntry cachedResult = getFromCache(cacheKey);
if (cachedResult != null) {
    log.info("从缓存获取路径规划结果: {}", cacheKey);
    long processingTime = System.currentTimeMillis() - startTime;
    return RouteResponse.success(cachedResult.getRouteResult(), processingTime);
}

// 调用高德API进行路径规划
RouteResponse.RouteResult routeResult = amapService.planRoute(request);

if (routeResult == null) {
    long processingTime = System.currentTimeMillis() - startTime;
    return RouteResponse.failure("路径规划失败, 请检查起终点地址是否正确", processingTime);
}

// 补充坐标信息
enrichRouteResult(routeResult, request);

// 缓存结果
putToCache(cacheKey, routeResult);

long processingTime = System.currentTimeMillis() - startTime;
log.info("路径规划成功: 距离={}米, 时长={}秒, 耗时={}ms",
        routeResult.getDistance(), routeResult.getDuration(), processingTime);

return RouteResponse.success(routeResult, processingTime);

} catch (IllegalArgumentException e) {
    long processingTime = System.currentTimeMillis() - startTime;
    log.warn("路径规划参数错误: {}", e.getMessage());
    return RouteResponse.failure(e.getMessage(), processingTime);
} catch (Exception e) {
    long processingTime = System.currentTimeMillis() - startTime;
    log.error("路径规划异常: {}", e.getMessage(), e);
    return RouteResponse.failure("系统异常, 请稍后重试", processingTime);
}
}

/**
 * 地理编码 - 地址转坐标
 *
 * @param address 地址
 * @param city 城市
 * @return 坐标响应

```

```

*/
public RouteResponse geocode(String address, String city) {
    long startTime = System.currentTimeMillis();

    try {
        if (!StringUtils.hasText(address)) {
            throw new IllegalArgumentException("地址不能为空");
        }

        String cityParam = StringUtils.hasText(city) ? city : defaultCity;
        String coordinate = amapService.geocode(address, cityParam);

        if (coordinate == null) {
            long processingTime = System.currentTimeMillis() - startTime;
            return RouteResponse.failure("地址解析失败, 请检查地址是否正确", processingTime);
        }

        // 解析坐标
        String[] coords = coordinate.split(",");
        if (coords.length != 2) {
            long processingTime = System.currentTimeMillis() - startTime;
            return RouteResponse.failure("坐标格式错误", processingTime);
        }

        RouteResponse.Coordinate coord = RouteResponse.Coordinate.builder()
            .longitude(Double.parseDouble(coords[0]))
            .latitude(Double.parseDouble(coords[1]))
            .address(address)
            .build();

        RouteResponse.RouteResult result = RouteResponse.RouteResult.builder()
            .origin(coord)
            .build();

        long processingTime = System.currentTimeMillis() - startTime;
        return RouteResponse.success(result, processingTime);

    } catch (Exception e) {
        long processingTime = System.currentTimeMillis() - startTime;
        log.error("地理编码异常: address={}, city={}, error={}", address, city, e.getMessage());
        return RouteResponse.failure("地理编码失败: " + e.getMessage(), processingTime);
    }
}

/**
 * 逆地理编码 - 坐标转地址
 *
 * @param longitude 经度
 * @param latitude 纬度
 * @return 地址响应

```

```

*/
public RouteResponse reverseGeocode(double longitude, double latitude) {
    long startTime = System.currentTimeMillis();

    try {
        validateCoordinate(longitude, latitude);

        String address = amapService.reverseGeocode(longitude, latitude);

        if (address == null) {
            long processingTime = System.currentTimeMillis() - startTime;
            return RouteResponse.failure("坐标解析失败, 请检查坐标是否正确", processingTime);
        }

        RouteResponse.Coordinate coord = RouteResponse.Coordinate.builder()
            .longitude(longitude)
            .latitude(latitude)
            .address(address)
            .build();

        RouteResponse.RouteResult result = RouteResponse.RouteResult.builder()
            .origin(coord)
            .build();

        long processingTime = System.currentTimeMillis() - startTime;
        return RouteResponse.success(result, processingTime);

    } catch (Exception e) {
        long processingTime = System.currentTimeMillis() - startTime;
        log.error("逆地理编码异常: lng={}, lat={}, error={}", longitude, latitude, e.getMessage());
        return RouteResponse.failure("逆地理编码失败: " + e.getMessage(), processingTime);
    }
}

/**
 * 验证和预处理请求参数
 */
private void validateAndPreprocessRequest(RouteRequest request) {
    if (!StringUtils.hasText(request.getOrigin())) {
        throw new IllegalArgumentException("起点不能为空");
    }

    if (!StringUtils.hasText(request.getDestination())) {
        throw new IllegalArgumentException("终点不能为空");
    }

    if (!StringUtils.hasText(request.getTransportMode())) {
        throw new IllegalArgumentException("交通方式不能为空");
    }
}

```

```
// 设置默认城市
if (!StringUtils.hasText(request.getCity())) {
    request.setCity(defaultCity);
}

// 设置默认策略
if (request.getStrategy() == null) {
    request.setStrategy(0);
}

// 设置时间戳
if (request.getTimestamp() == null) {
    request.setTimestamp(System.currentTimeMillis());
}
}

/**
 * 验证坐标有效性
 */
private void validateCoordinate(double longitude, double latitude) {
    if (longitude < -180 || longitude > 180) {
        throw new IllegalArgumentException("经度必须在-180到180之间");
    }

    if (latitude < -90 || latitude > 90) {
        throw new IllegalArgumentException("纬度必须在-90到90之间");
    }
}

/**
 * 补充路径结果信息
 */
private void enrichRouteResult(RouteResponse.RouteResult routeResult, RouteRequest request) {
    try {
        // 解析起点坐标
        String originCoord = getCoordinateString(request.getOrigin(), request.getCity());
        if (originCoord != null) {
            String[] coords = originCoord.split(",");
            if (coords.length == 2) {
                RouteResponse.Coordinate origin = RouteResponse.Coordinate.builder()
                    .longitude(Double.parseDouble(coords[0]))
                    .latitude(Double.parseDouble(coords[1]))
                    .address(request.getOrigin())
                    .build();
                routeResult.setOrigin(origin);
            }
        }

        // 解析终点坐标
        String destCoord = getCoordinateString(request.getDestination(), request.getCity());
    }
}
```

```

        if (destCoord != null) {
            String[] coords = destCoord.split(",");
            if (coords.length == 2) {
                RouteResponse.Coordinate destination = RouteResponse.Coordinate.builder()
                    .longitude(Double.parseDouble(coords[0]))
                    .latitude(Double.parseDouble(coords[1]))
                    .address(request.getDestination())
                    .build();
                routeResult.setDestination(destination);
            }
        }

    } catch (Exception e) {
        log.warn("补充路径结果信息失败: {}", e.getMessage());
    }
}

/**
 * 获取坐标字符串
 */
private String getCoordinateString(String location, String city) {
    if (RouteRequest.isValidCoordinate(location)) {
        return location;
    }
    return amapService.geocode(location, city);
}

/**
 * 生成缓存键
 */
private String generateCacheKey(RouteRequest request) {
    StringBuilder key = new StringBuilder();
    key.append(request.getOrigin()).append("|");
    key.append(request.getDestination()).append("|");
    key.append(request.getTransportMode()).append("|");
    key.append(request.getStrategy()).append("|");
    if (request.getFormattedWaypoints() != null) {
        key.append(request.getFormattedWaypoints()).append("|");
    }
    key.append(request.getCity());
    return key.toString();
}

/**
 * 从缓存获取结果
 */
private CacheEntry getFromCache(String key) {
    CacheEntry entry = routeCache.get(key);
    if (entry != null && !entry.isExpired()) {
        return entry;
    }
}

```

```

    }
    if (entry != null) {
        routeCache.remove(key); // 移除过期缓存
    }
    return null;
}

/**
 * 将结果放入缓存
 */
private void putToCache(String key, RouteResponse.RouteResult result) {
    long expireTime = System.currentTimeMillis() + TimeUnit.MINUTES.toMillis(cacheExpireMinutes);
    routeCache.put(key, new CacheEntry(result, expireTime));
}

/**
 * 缓存条目
 */
private static class CacheEntry {
    private final RouteResponse.RouteResult routeResult;
    private final long expireTime;

    public CacheEntry(RouteResponse.RouteResult routeResult, long expireTime) {
        this.routeResult = routeResult;
        this.expireTime = expireTime;
    }

    public RouteResponse.RouteResult getRouteResult() {
        return routeResult;
    }

    public boolean isExpired() {
        return System.currentTimeMillis() > expireTime;
    }
}

/**
 * 清理过期缓存（可以通过定时任务调用）
// ===== 文件: java-backend/src/main/java/com/ljxz/cycling/service/AmapService.java | 技术栈: Java + Spring Boot (Se...
package com.ljxz.cycling.service;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.ljxz.cycling.entity.RouteRequest;
import com.ljxz.cycling.entity.RouteResponse;
import lombok.extern.slf4j.Slf4j;
import org.apache.http.HttpEntity;
import org.apache.http.client.config.RequestConfig;
import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.client.methods.HttpGet;

```

```

import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.util.EntityUtils;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.util.StringUtils;

import java.io.IOException;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;

/**
 * 高德地图API服务
 *
 * @author LJXZ Team
 * @version 1.0.0
 */
@Slf4j
@Service
public class AmapService {

    @Value("${amap.web-api-key}")
    private String webApiKey;

    @Value("${amap.base-url}")
    private String baseUrl;

    @Value("${amap.timeout:10000}")
    private int timeout;

    @Value("${amap.retry-count:3}")
    private int retryCount;

    private final ObjectMapper objectMapper = new ObjectMapper();
    private final CloseableHttpClient httpClient;

    public AmapService() {
        // 配置HTTP客户端
        RequestConfig requestConfig = RequestConfig.custom()
            .setConnectTimeout(timeout)
            .setSocketTimeout(timeout)
            .setConnectionRequestTimeout(timeout)
            .build();

        this.httpClient = HttpClients.custom()
            .setDefaultRequestConfig(requestConfig)
            .setMaxConnTotal(100)
            .setMaxConnPerRoute(20)

```

```

        .build();
    }

    /**
     * 地理编码 - 将地址转换为坐标
     *
     * @param address 地址
     * @param city 城市
     * @return 坐标字符串 "longitude, latitude"
     */
    public String geocode(String address, String city) {
        try {
            String url = buildGeocodeUrl(address, city);
            String response = executeHttpRequest(url);
            return parseGeocodeResponse(response);
        } catch (Exception e) {
            log.error("地理编码失败: address={}, city={}, error={}", address, city, e.getMessage());
            return null;
        }
    }

    /**
     * 逆地理编码 - 将坐标转换为地址
     *
     * @param longitude 经度
     * @param latitude 纬度
     * @return 地址描述
     */
    public String reverseGeocode(double longitude, double latitude) {
        try {
            String url = buildReverseGeocodeUrl(longitude, latitude);
            String response = executeHttpRequest(url);
            return parseReverseGeocodeResponse(response);
        } catch (Exception e) {
            log.error("逆地理编码失败: lng={}, lat={}, error={}", longitude, latitude, e.getMessage());
            return null;
        }
    }

    /**
     * 路径规划
     *
     * @param request 路径规划请求
     * @return 路径规划结果
     */
    public RouteResponse.RouteResult planRoute(RouteRequest request) {
        try {
            // 转换起终点为坐标
            String originCoord = convertToCoordinate(request.getOrigin(), request.getCity());
            String destCoord = convertToCoordinate(request.getDestination(), request.getCity());

```

```

        if (originCoord == null || destCoord == null) {
            log.error("起终点坐标转换失败: origin={}, destination={}", request.getOrigin(), request.getDestination());
            return null;
        }

        // 构建路径规划URL
        String url = buildRouteUrl(request, originCoord, destCoord);
        String response = executeHttpRequest(url);
        return parseRouteResponse(response, request.getTransportMode());

    } catch (Exception e) {
        log.error("路径规划失败: request={}, error={}", request, e.getMessage());
        return null;
    }
}

/**
 * 将地址或坐标转换为标准坐标格式
 */
private String convertToCoordinate(String location, String city) {
    if (StringUtils.isEmpty(location)) {
        return null;
    }

    // 如果已经是坐标格式, 直接返回
    if (RouteRequest.isValidCoordinate(location)) {
        return location;
    }

    // 否则进行地理编码
    return geocode(location, city);
}

/**
 * 构建地理编码URL
 */
private String buildGeocodeUrl(String address, String city) {
    StringBuilder url = new StringBuilder(baseUrl + "/geocode/geo");
    url.append("?key=").append(webApiKey);
    url.append("&address=").append(URLEncoder.encode(address, StandardCharsets.UTF_8));
    if (StringUtils.hasText(city)) {
        url.append("&city=").append(URLEncoder.encode(city, StandardCharsets.UTF_8));
    }
    url.append("&output=json");
    return url.toString();
}

/**
 * 构建逆地理编码URL

```

```

*/
private String buildReverseGeocodeUrl(double longitude, double latitude) {
    StringBuilder url = new StringBuilder(baseUrl + "/geocode/regeo");
    url.append("?key=").append(webApiKey);
    url.append("&location=").append(longitude).append(", ").append(latitude);
    url.append("&output=json");
    url.append("&radius=1000");
    url.append("&extensions=base");
    return url.toString();
}

/**
 * 构建路径规划URL
 */
private String buildRouteUrl(RouteRequest request, String origin, String destination) {
    String endpoint = getRouteEndpoint(request.getTransportMode());
    StringBuilder url = new StringBuilder(baseUrl + endpoint);
    url.append("?key=").append(webApiKey);
    url.append("&origin=").append(origin);
    url.append("&destination=").append(destination);

    // 添加途经点
    if (request.getFormattedWaypoints() != null) {
        url.append("&waypoints=").append(URLEncoder.encode(request.getFormattedWaypoints(), StandardCharsets...
    }

    // 添加策略
    url.append("&strategy=").append(request.getStrategy());

    // 添加其他参数
    url.append("&output=json");
    url.append("&extensions=all");

    if (StringUtils.hasText(request.getCity())) {
        url.append("&city=").append(URLEncoder.encode(request.getCity(), StandardCharsets.UTF_8));
    }

    return url.toString();
}

/**
 * 获取路径规划API端点
 */
private String getRouteEndpoint(String transportMode) {
    switch (transportMode) {
        case "driving":
            return "/direction/driving";
        case "walking":
            return "/direction/walking";
        case "transit":

```

```

        return "/direction/transit/integrated";
    case "riding":
        return "/direction/bicycling";
    default:
        return "/direction/driving";
    }
}

/**
 * 执行HTTP请求
 */
private String executeHttpRequest(String url) throws IOException {
    HttpGet httpGet = new HttpGet(url);
    httpGet.setHeader("User-Agent", "LJXZ-Cycling-Route/1.0");

    for (int i = 0; i < retryCount; i++) {
        try (CloseableHttpResponse response = httpClient.execute(httpGet)) {
            HttpEntity entity = response.getEntity();
            if (entity != null) {
                String result = EntityUtils.toString(entity, StandardCharsets.UTF_8);
                log.debug("高德API响应: {}", result);
                return result;
            }
        } catch (IOException e) {
            log.warn("HTTP请求失败, 第{}次重试: {}", i + 1, e.getMessage());
            if (i == retryCount - 1) {
                throw e;
            }
            try {
                Thread.sleep(1000 * (i + 1)); // 递增延迟
            } catch (InterruptedException ie) {
                Thread.currentThread().interrupt();
                throw new IOException("请求被中断", ie);
            }
        }
    }
    throw new IOException("HTTP请求失败, 已重试" + retryCount + "次");
}

// ===== 文件: vue-cycling-app/src/views/Home.vue | 技术栈: Vue 3 + JavaScript =====
<template>
  <!-- Hero 区域 -->
  <section class="hero">
    <div class="hero-content">
      <h1>探索城市周边最美骑行路线</h1>
      <p>精选风景各异的骑行路线, 无论初学者还是资深骑友, 都能找到属于自己的风景</p>
      <div class="hero-actions">
        <a href="#routes" class="cta-button">探索路线</a>
        <router-link to="/route-planning" class="cta-button-outline">
          <i class="fas fa-route"></i> 智能规划
        </router-link>
      </div>
    </div>
  </section>
</template>

```

```

        </router-link>
    </div>
    <div class="hero-stats-mini">
        <span><strong>28</strong> 精选路线</span>
        <span><strong>12,000+</strong> 活跃骑友</span>
        <span><strong>86,000km</strong> 累计骑行</span>
    </div>
</div>
</section>

<!-- 精选骑行路线 -->
<main class="container">
    <section id="routes">
        <h2 class="section-title">精选骑行路线</h2>
        <p class="section-subtitle">精选非遗文化与自然风光融合的骑行体验，每条路线都有独特的故事</p>

        <div class="route-container">
            <div class="route-card" v-for="(route, index) in featuredRoutes" :key="index">
                <div class="route-image">
                    
                </div>
                <div class="route-content">
                    <div class="route-header-row">
                        <span :class="['difficulty', route.difficultyClass]">{{ route.difficulty }}</span>
                        <span class="route-tag" v-if="route.tag"><i :class="route.tagIcon"></i> {{ route.tag }}</span>
                    </div>
                    <h3>{{ route.name }}</h3>
                    <p>{{ route.description }}</p>
                    <div class="route-stats">
                        <div class="stat-item">
                            <i class="fas fa-route"></i>
                            <h4>距离</h4>
                            <p>{{ route.distance }}</p>
                        </div>
                        <div class="stat-item">
                            <i class="far fa-clock"></i>
                            <h4>时间</h4>
                            <p>{{ route.time }}</p>
                        </div>
                        <div class="stat-item">
                            <i class="fas fa-mountain"></i>
                            <h4>海拔</h4>
                            <p>{{ route.elevation }}</p>
                        </div>
                    </div>
                    <div class="route-actions">
                        <button class="btn btn-outline" @click="openRouteDetail(route)">
                            <i class="fas fa-book-open"></i> 路线详情
                        </button>
                        <router-link to="/route-planning" class="btn btn-primary">

```

```

        <i class="fas fa-map-marked-alt"></i> 地图规划
    </router-link>
</div>
</div>
</div>

<!-- 探索更多卡片 -->
<router-link to="/intangible-heritage-map" class="route-card more-routes-card">
    <div class="more-routes-content">
        <div class="more-icon">
            <i class="fas fa-landmark"></i>
        </div>
        <h3>探索更多路线</h3>
        <p>发现10+非遗乡村骑行线路，从太行水镇到磧口古镇，每条路线都承载着独特的文化记忆。</p>
        <div class="more-stats">
            <span><strong>10</strong>个非遗乡村</span>
            <span><strong>3</strong>条主题路线</span>
            <span><strong>1</strong>场文化之旅</span>
        </div>
        <span class="btn btn-full">
            进入非遗骑行地图 <i class="fas fa-arrow-right"></i>
        </span>
    </div>
</router-link>
</div>
</section>
</main>

<!-- 路线详情弹窗（非地图入口） -->
<div v-if="selectedRoute" class="modal-overlay" @click.self="selectedRoute = null">
    <div class="modal-content route-detail-modal">
        <div class="modal-header">
            
            <button class="modal-close" @click="selectedRoute = null">
                <i class="fas fa-times"></i>
            </button>
            <div class="modal-header-info">
                <span :class="['difficulty', selectedRoute.difficultyClass]">{{ selectedRoute.difficulty }}</span>
                <h2>{{ selectedRoute.name }}</h2>
            </div>
        </div>
        <div class="modal-body">
            <div class="detail-stats-bar">
                <div class="detail-stat">
                    <i class="fas fa-route"></i>
                    <span>{{ selectedRoute.distance }}</span>
                </div>
                <div class="detail-stat">
                    <i class="far fa-clock"></i>
                    <span>{{ selectedRoute.time }}</span>
                </div>
            </div>
        </div>
    </div>
</div>

```

```

    </div>
    <div class="detail-stat">
      <i class="fas fa-mountain"></i>
      <span>{{ selectedRoute.elevation }}</span>
    </div>
    <div class="detail-stat">
      <i class="fas fa-star"></i>
      <span>{{ selectedRoute.rating }}</span>
    </div>
  </div>
  <div class="detail-section">
    <h4><i class="fas fa-align-left"></i> 路线简介</h4>
    <p>{{ selectedRoute.fullDescription }}</p>
  </div>
  <div class="detail-section">
    <h4><i class="fas fa-bullseye"></i> 沿途亮点</h4>
    <ul>
      <li v-for="(highlight, idx) in selectedRoute.highlights" :key="idx">
        <i class="fas fa-check-circle"></i> {{ highlight }}
      </li>
    </ul>
  </div>
  <div class="detail-section">
    <h4><i class="fas fa-lightbulb"></i> 骑行贴士</h4>
    <p class="tips-box">{{ selectedRoute.tips }}</p>
  </div>
</div>
<div class="modal-footer">
  <button class="btn btn-secondary" @click="selectedRoute = null">关闭</button>
  <router-link to="/route-planning" class="btn btn-primary" @click="selectedRoute = null">
    <i class="fas fa-location-arrow"></i> 去地图规划此路线
  </router-link>
</div>
</div>
</div>

<!-- 平台数据统计 -->
<section class="stats-showcase">
  <div class="container">
    <h2 class="section-title">骑行者社区的数据</h2>
    <div class="stats-grid">
      <div class="stat-card" v-for="(item, index) in statsData" :key="index">
        <div class="stat-icon"><i :class="'fas ' + item.icon"></i></div>
        <div class="stat-number">{{ item.number }}</div>
        <div class="stat-unit">{{ item.unit }}</div>
        <div class="stat-label">{{ item.label }}</div>
      </div>
    </div>
  </div>
</section>

```

```

<!-- 特色功能探索 -->
<section class="features-explore">
  <div class="container">
    <h2 class="section-title">特色功能探索</h2>
    <p class="section-subtitle">不止于路线，我们为你提供全方位的骑行体验</p>

    <div class="features-grid">
      <router-link to="/intangible-heritage-map" class="feature-card">
        <div class="feature-icon heritage">
          <i class="fas fa-landmark"></i>
        </div>
        <h3>非遗骑行地图</h3>
        <p>穿越千年文脉，用车轮丈量非遗之美。精选10个非遗乡村骑行线路。</p>
        <span class="feature-link">探索非遗路线 <i class="fas fa-arrow-right"></i></span>
      </router-link>

      <router-link to="/vr" class="feature-card">
        <div class="feature-icon vr">
          <i class="fas fa-vr-cardboard"></i>
        </div>
        <h3>VR灵境</h3>
        <p>足不出户，身临其境。360° 沉浸式预览沿途风景，提前规划你的骑行。</p>
        <span class="feature-link">进入VR体验 <i class="fas fa-arrow-right"></i></span>
      </router-link>

      <router-link to="/equipment" class="feature-card">
        <div class="feature-icon equipment">
          <i class="fas fa-bicycle"></i>
        </div>
        <h3>装备推荐</h3>
        <p>基于大数据分析的骑行装备选购指南，从入门到专业，一站配齐。</p>
        <span class="feature-link">查看装备 <i class="fas fa-arrow-right"></i></span>
      </router-link>

      <router-link to="/route-planning" class="feature-card">
        <div class="feature-icon planning">
          <i class="fas fa-route"></i>
        </div>
        <h3>智能路线规划</h3>
        <p>基于高德地图API，输入起点终点，自动生成最优骑行路线。</p>
        <span class="feature-link">开始规划 <i class="fas fa-arrow-right"></i></span>
      </router-link>
    </div>
  </div>
</section>

<!-- 为什么选择我们 -->
<section class="why-choose-us">
  <div class="container">

```

```

<div class="value-content">
  <div class="value-text">
    <h2 class="section-title left">为什么选择我们</h2>
    <p class="value-desc">我们不仅提供路线，更致力于打造完整的骑行生态系统</p>

    <div class="value-list">
      <div class="value-item">
        <div class="value-icon"><i class="fas fa-check-circle"></i></div>
        <div>
          <h4>精选实测路线</h4>
          <p>每条路线均经过实地骑行验证，附详细路况、海拔、设施信息</p>
        </div>
      </div>
      <div class="value-item">
        <div class="value-icon"><i class="fas fa-check-circle"></i></div>
        <div>
          <h4>智能路线规划</h4>
          <p>接入高德地图API，支持任意两点间的骑行路线规划与导航</p>
        </div>
      </div>
      <div class="value-item">
        <div class="value-icon"><i class="fas fa-check-circle"></i></div>
        <div>
          <h4>文化+骑行融合</h4>
          <p>独创非遗骑行地图，让骑行不仅是运动，更是一场文化之旅</p>
        </div>
      </div>
      <div class="value-item">
        <div class="value-icon"><i class="fas fa-check-circle"></i></div>
        <div>
          <h4>装备数据支持</h4>
          <p>基于后端数据分析的装备推荐与价格趋势，让选购更明智</p>
        </div>
      </div>
    </div>

    <div class="value-visual">
      
      <div class="value-badge">
        <i class="fas fa-award"></i>
        <span>最佳骑行平台</span>
      </div>
    </div>
  </div>
</section>

<!-- 底部 CTA -->
<section class="bottom-cta">

```

```

<div class="container">
  <h2>准备好出发了吗? </h2>
  <p>加入我们的骑行社区，发现更多精彩路线与志同道合的骑友</p>
  <div class="cta-actions">
    <router-link to="/register" class="cta-button">免费注册</router-link>
    <router-link to="/route-planning" class="cta-button-outline">
      <i class="fas fa-route"></i> 先试规划
    </router-link>
  </div>
</div>
</section>
</template>

<script setup>
import { ref } from 'vue'

const imageLoaded = (event) => {
  event.target.parentElement.classList.add('loaded');
};

const selectedRoute = ref(null)

const openRouteDetail = (route) => {
  selectedRoute.value = route
}

const featuredRoutes = ref([
  {
    name: '太行水镇非遗体验线',
    difficulty: '初级',
    difficultyClass: 'easy',
    tag: '非遗民俗',
    tagIcon: 'fas fa-landmark',
    image: '/media/2.jpg',
    description: '串联太行水镇、恋乡古村与老磨坊，体验河北民俗文化与农耕非遗...',
    distance: '18 公里',
    time: '2-3 小时',
    elevation: '80m',
    rating: '★★★★★',
    fullDescription: '太行水镇非遗体验线是一条融合自然风光与传统文化的热门骑行路线。从太行水镇出发，途经保存完好的恋乡古村，最终抵达传统老',
    highlights: ['漫步太行水镇古街，品尝地方特色美食', '参观恋乡古村石板路与石头房建筑', '亲身体验老磨坊传统石磨研磨', '沿易水河畔骑行，欣赏',
    tips: '建议清晨8-9点出发，可先在水镇内享用早餐后开始骑行。全程设有多个休息点，记得携带水壶和防晒用品。'
  },
  {
    name: '碛口古镇晋商文化线',
    difficulty: '中级',
    difficultyClass: 'moderate',
    tag: '晋商文化',
    tagIcon: 'fas fa-history',
    image: '/media/17.jpg',

```

```

description: '穿越碛口古镇、黑龙庙与黄土窑洞群, 感受晋商文化与黄河文明...',
distance: '32 公里',
time: '4-5 小时',
elevation: '320m',
rating: '★★★★★',
fullDescription: '碛口古镇晋商文化线带你穿越黄河岸边的明清古镇, 感受晋商文化的厚重底蕴。路线从古镇客栈群出发, 途经黑龙庙戏台, 穿越黄土
highlights: ['探访明清时期晋商古客栈与商铺', '在黑龙庙戏台聆听地方戏曲', '参观依山而建的黄土窑洞群民居', '俯瞰黄河壮丽景色'],
tips: '古镇内多为石板路, 骑行时请注意路面状况并减速慢行。建议安排充足时间, 可考虑在古镇住宿一晚体验夜景。'
},
{
name: '易水湖环湖栈道线',
difficulty: '初级',
difficultyClass: 'easy',
tag: '山水风光',
tagIcon: 'fas fa-water',
image: '/media/14.jpg',
description: '沿湖而建的景观骑行栈道, 湖光山色尽收眼底, 适合休闲骑行...',
distance: '15 公里',
time: '2 小时',
elevation: '30m',
rating: '★★★★☆',
fullDescription: '易水湖环湖栈道线是专为骑行和步行设计的景观道路, 沿湖蜿蜒而行。骑行其间, 碧波荡漾的湖水与连绵的太行山交相辉映, 沿途设
highlights: ['沿湖骑行欣赏太行山水倒影', '在湖心观景台打卡拍照', '了解荆轲刺秦的易水文化典故', '途经湿地公园观察水鸟生态'],
tips: '栈道部分路段临水, 请注意安全。建议携带防晒用品和水壶, 春夏季节风景最佳。'
},
{
name: '黄土窑洞民居探索线',
difficulty: '中级',
difficultyClass: 'moderate',
tag: '民居技艺',
tagIcon: 'fas fa-home',
image: '/media/26.jpg',
description: '探访依山而建的窑洞村落, 体验黄土高原独特的居住智慧与民俗风情...',
distance: '25 公里',
time: '3.5 小时',
elevation: '280m',
rating: '★★★★☆',
fullDescription: '黄土窑洞民居探索线深入黄土高原腹地, 探访保存完好的窑洞村落。窑洞作为黄土高原特有的民居形式, 具有冬暖夏凉的天然优势。
highlights: ['参观层层叠叠的窑洞群建筑', '了解窑洞开凿与建造技艺', '体验窑洞民宿的独特居住感受', '品尝黄土高原特色农家美食'],
tips: '部分路段有爬坡, 建议提前检查车辆变速系统。窑洞住宿体验独特但设施相对简单, 建议提前了解住宿条件。'
}
])

const statsData = ref([
  { icon: 'fa-road', number: '86', unit: '万公里', label: '累计骑行里程' },
  { icon: 'fa-map-marked-alt', number: '28', unit: '条', label: '精选骑行路线' },
  { icon: 'fa-users', number: '12,000', unit: '+', label: '活跃骑行用户' },
  { icon: 'fa-mountain', number: '158', unit: '万米', label: '累计爬升海拔' }
])
</script>

```